

Team 11: Sentiment Analysis and Rating Prediction

Submitted by

Skanda P (Roll No.: CS24BT045)

Krish Gupta (Roll No.: CS24BT038)

Shashikant Patel (Roll No.: IS24BM045)

Tanveer Rafiq Shaik (Roll No.: MC24BT014)

Palakurthy Shyam Praveen (Roll No.: CS24BT039)

Under the Supervision of
Deepan Raj



Department of Computer Science and Engineering
Indian Institute of Technology Dharwad

April, 2026

Abstract

This report presents a compact summary of the study on sentiment classification and rating prediction for recipe reviews. Three classifier models were examined for sentiment analysis: Logistic Regression, Linear Support Vector Classification (Linear SVC), and LSTM with word embeddings. The final system emphasises **Linear SVC**, because it is well matched to sparse TF-IDF features, remains efficient on CPU, and achieves the strongest overall balance on the minority Negative and Neutral classes among the directly comparable three-class models. A major strength of the SVM pipeline is its **feature engineering design**: a 15,000- dimensional delta TF-IDF representation is fused with 25 engineered meta-features capturing polarity, subjectivity, linguistic structure, reviewer intent, and sentence-level sentiment dynamics. The results section prioritises the reported metrics for all models and replaces image-based SVM results with uniform tables.

Contents

1	Introduction	1
1.1	Objectives	1
2	Dataset Description	2
3	General Model Comparison	5
3.1	Model Comparison Study	5
3.1.1	Sentiment Classification Models	5
3.1.2	Rating Prediction Models	8
3.1.3	Algorithm Selection Justification	10
3.1.4	Regression Model Selection Justification	11
4	Model-wise Analysis	13
4.1	Logistic Regression	13
4.1.1	Preprocessing and Implementation	13
4.1.2	Results	14
4.2	Linear SVC	16
4.2.1	Preprocessing and Implementation	16
4.2.2	Results	18
4.3	LSTM with Word Embeddings	21
4.3.1	Preprocessing and Implementation	21
4.3.2	Results	21
4.4	XGBoost	23
4.4.1	Preprocessing and Implementation	23

4.4.2	Results	23
4.5	Linear & Ridge Regression	25
4.5.1	Preprocessing and Implementation	25
4.5.2	Results	25
5	Conclusion	28
	References	29

List of Tables

2.1	Core attributes used in the modelling pipeline	3
2.2	Dataset split used for the reported experiments	4
3.1	Comparative study of candidate algorithms for sentiment clas- sification	6
3.2	Summary of reported sentiment-classification performance . . .	8
3.3	Comparative study of candidate algorithms for rating prediction	8
3.4	Summary of reported rating-prediction performance	10
4.1	Hyperparameter grid and rationale for Logistic Regression . .	14
4.2	Logistic Regression accuracy across data splits	14
4.3	Logistic Regression classification report on the test set	15
4.4	Summary of the feature engineering pipeline used for Linear SVC	17
4.5	Grouped view of the 25 engineered meta-features used in the SVM model	18
4.6	Test-set performance summary for SVM and baseline models .	19
4.7	Linear SVC split-wise performance before and after tuning . .	19
4.8	Optuna-tuned Linear SVC classification report on the test set	19
4.9	Classification report for LSTM	21
4.10	Performance metrics for LSTM	22
4.11	Reported XGBoost performance for rating prediction	23
4.12	Performance comparison of Linear and Ridge Regression . . .	25

Chapter 1

Introduction

Online recipe platforms contain large volumes of short, subjective, and often informal user feedback. Automatically analysing this feedback is useful for understanding user satisfaction, identifying failure cases in recipes, and predicting ratings from textual evidence. In this project, the review text is used for two related tasks: *sentiment classification* and *rating prediction*.

The sentiment task maps each review to one of three labels—*Negative*, *Neutral*, or *Positive*. The rating task predicts the numerical star score assigned by the reviewer. Because the corpus is strongly skewed toward positive reviews, the analysis must go beyond plain accuracy and give substantial importance to class-balanced metrics such as Macro-F1 and balanced accuracy.

1.1 Objectives

The main objectives of the project are:

- compare multiple sentiment models on the same review corpus,
- design a strong feature representation for sparse text classification,
- prioritise results on minority classes rather than majority-class accuracy alone, and
- identify a practical final pipeline for both classification and rating prediction.

Chapter 2

Dataset Description

The experiments use the **Recipe Reviews and User Feedback Dataset** from the UCI Machine Learning Repository [1]. After removing invalid rows with `stars = 0` and entries with missing review text, the final working corpus contains **16,484 reviews**. Each review includes free-form text together with structured metadata such as rating, reviewer reputation, and vote statistics. This combination makes the dataset suitable for the two linked tasks studied in the report: *sentiment classification* from review language and *rating prediction* from both text and structured signals.

The dataset is also useful because it contains both subjective and behavioural information. The review text captures satisfaction, disappointment, and mixed opinions in natural language, while metadata such as community feedback and user reputation provide supporting context. In practice, this means the corpus is rich enough to test simple linear baselines, feature-engineered sparse models, and more expressive sequence or tree-based methods within a single shared benchmark.

Table 2.1: Core attributes used in the modelling pipeline

Feature	Description	Type
<code>text</code>	Raw review text written by the user	Text
<code>stars</code>	User-given rating on a 1–5 scale	Integer
<code>best_score</code>	Aggregate review quality indicator	Float
<code>user_reputation</code>	Reputation score associated with the reviewer	Integer
<code>reply_count</code>	Number of replies received by the review	Integer
<code>thumbs_up,</code> <code>thumbs_down</code>	Community feedback counts	Integer
<code>created_at</code>	Review timestamp used for temporal features	Integer

For sentiment classification, the star ratings are mapped as follows:

$$1\text{--}2 \text{ stars} \rightarrow \textit{Negative}, \quad 3 \text{ stars} \rightarrow \textit{Neutral}, \quad 4\text{--}5 \text{ stars} \rightarrow \textit{Positive}.$$

This mapping preserves the ordinal interpretation of the original ratings while turning the problem into a three-class sentiment task. However, the resulting label distribution is strongly imbalanced: Positive reviews dominate the corpus, whereas Neutral and Negative reviews occur much less frequently. This imbalance is important because a model can achieve deceptively high overall accuracy simply by favouring the majority class.

To make this skew explicit, Figure 2.1 presents the uploaded exploratory distributions for both derived sentiment labels and raw star ratings. The left panel confirms that the Positive class overwhelmingly dominates the sentiment mapping, while the right panel shows that the original ratings are heavily concentrated at 5 stars, followed by 4 stars, with very few low-rated reviews. This pattern motivates the evaluation strategy used in the rest of the report: Macro-F1 and minority-class behaviour are emphasised for classification, and RMSE, R^2 , and tolerance-based accuracy are used to give a more reliable picture of rating-prediction quality.

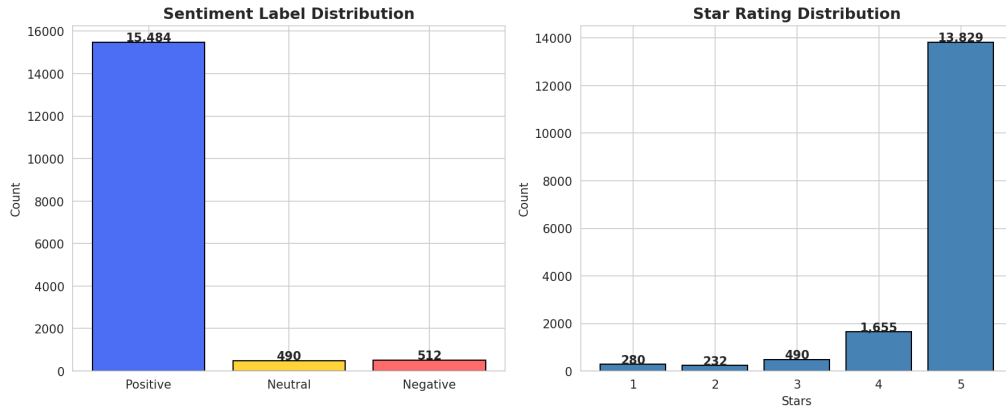


Figure 2.1: Exploratory distributions of sentiment labels and star ratings in the cleaned review dataset

The common evaluation protocol uses a stratified 70/15/15 train-validation-test split so that class proportions remain stable across all subsets. This is especially important for the sentiment task because the minority classes would otherwise fluctuate substantially across partitions, making validation and test performance less reliable to interpret.

Table 2.2: Dataset split used for the reported experiments

Split	Proportion	Approx. Samples	Purpose
Training	70%	$\sim 11,539$	Fit preprocessors and train models
Validation	15%	$\sim 2,472$	Hyperparameter tuning and model selection
Test	15%	$\sim 2,473$	Final unbiased evaluation

Chapter 3

General Model Comparison

This chapter first compares the candidate models qualitatively and then consolidates the reported quantitative results in two task-specific summary tables. Separating the classifier and regression summaries removes the many not-applicable cells that appeared in the earlier mixed table and makes each comparison easier to interpret.

3.1 Model Comparison Study

Before selecting the final algorithms, a comparative analysis was carried out separately for the three sentiment-classification models and the three rating-prediction regressors. This study goes beyond reported metrics and also considers implementation complexity, computational cost, interpretability, and practical suitability for the project dataset.

3.1.1 Sentiment Classification Models

For sentiment classification, the comparison must account for the nature of the feature space: the review corpus is represented either through sparse TF-IDF vectors or through sequence-based embeddings, and the dataset is strongly imbalanced toward Positive reviews. Table 3.1 therefore compares Logistic Regression, Linear SVC, and LSTM not only in terms of learning

capacity but also with respect to their fitness for sparse text, minority-class handling, and deployment cost.

Table 3.1: Comparative study of candidate algorithms for sentiment classification

Criterion	Logistic sion	Regres- sion	SVM / SVC	Linear	LSTM + Embeddings	Word
Difficulty Level	Simple. linear separability and uses a well-understood optimisation framework based on log-loss.	Assumes separability and uses a well-understood optimisation framework based on log-loss.	Moderate. hinge-loss maximisation; mathematically elegant and particularly effective in high-dimensional sparse spaces.	Uses margin elegantly and particularly effective in high-dimensional sparse spaces.	High. Requires understanding of recurrent networks, backpropagation through time, sequence padding, and embedding representations.	Requires understanding of recurrent networks, backpropagation through time, sequence padding, and embedding representations.
Scope	Binary or multi-class classification on approximately linear boundaries; works well with TF-IDF features.	Binary or multi-class classification on approximately linear boundaries; works well with TF-IDF features.	Excellent for high-dimensional sparse data such as TF-IDF; especially strong when class boundaries benefit from maximum-margin separation.	Excellent for high-dimensional sparse data such as TF-IDF; especially strong when class boundaries benefit from maximum-margin separation.	Best suited to sequence modelling problems where word order and local context materially affect sentiment.	Best suited to sequence modelling problems where word order and local context materially affect sentiment.
Code Availability	LogisticRegression in scikit-learn; trivial to deploy and extensively documented.	LogisticRegression in scikit-learn; trivial to deploy and extensively documented.	LinearSVC with a production-grade LIBLINEAR back-end; mature and highly optimised.	LinearSVC with a production-grade LIBLINEAR back-end; mature and highly optimised.	torch.nn.LSTM and Keras LSTM; widely available but typically requires more custom training code and embedding management.	torch.nn.LSTM and Keras LSTM; widely available but typically requires more custom training code and embedding management.

Continued on next page

Table 3.1: Comparative study of candidate algorithms for sentiment classification (continued)

Criterion	Logistic sion	Regres- sion	SVM / SVC	Linear	LSTM + Embeddings	Word
Resource Re- quirements	Very low. Trains quickly on CPU and is easy to tune.		Low to moderate. Faster than kernel SVMs and still practical on CPU for the present dataset size.		High. Training scales with sequence length and batch size; GPU support is typically desirable for efficient iteration.	
Interpretability	High. Coefficients directly indicate which features push predictions toward each class.		Moderate. Linear coefficients remain interpretable, but the margin-based decision function is less intuitive than Logistic Regression probabilities.		Low. Internal hidden states and embeddings provide limited transparency, making the model comparatively black-box.	
Scalability	Moderate. Efficient on sparse data, but performance may plateau when class boundaries become more complex.		High for sparse linear data. LinearSVC handles the 15,000-dimensional feature space efficiently and scales well on CPU.		High in representational capacity, but cost-effective only when sufficient data and compute are available.	

The classifier comparison shows a clear trade-off. Logistic Regression is the simplest and most interpretable baseline, but it is weaker on the difficult Neutral class. LSTM offers the richest modelling capacity, yet that capacity comes with higher compute cost and weaker comparability to the shared sparse pipeline. Linear SVC occupies the most practical middle ground: it keeps the computational efficiency of a linear model while improving minority-class separation through margin-based learning.

Table 3.2: Summary of reported sentiment-classification performance

Model	Accuracy	Macro-F1
Logistic Regression	0.9523	0.6300
Linear SVC (tuned)	0.9531	0.6439
LSTM + Word Embeddings	0.7297	0.7333

3.1.2 Rating Prediction Models

For rating prediction, the decision boundary is no longer categorical but continuous, so the emphasis shifts from class-wise F1 behaviour to error stability, non-linear modelling ability, and robustness under correlated features. Table 3.3 therefore compares the two linear baselines with XGBoost along the same broad dimensions used for the classifier study.

Table 3.3: Comparative study of candidate algorithms for rating prediction

Criterion	Linear Regression	Ridge Regression	XGBoost
Difficulty Level	Simple. Based on ordinary least squares and easy to analyse mathematically.	Simple to moderate. Extends Linear Regression with an L2 penalty and one main regularisation hyperparameter.	Moderate to high. Uses stage-wise gradient-boosted trees and a larger interacting hyperparameter space.
Scope	Useful as a fast baseline when the relationship between features and ratings is approximately linear.	Well suited to high-dimensional or correlated feature spaces where co-efficient shrinkage improves stability.	Strong for heterogeneous tabular inputs and non-linear interactions between metadata and text-derived features.

Continued on next page

Table 3.3: Comparative study of candidate algorithms for rating prediction (continued)

Criterion		Linear Regression	Ridge Regression	XGBoost
Code availability	Avail-	LinearRegression in scikit-learn; trivial to train and deploy.	Ridge in scikit-learn; equally mature and easy to reproduce.	XGBRegressor; widely used in practice and supported by a mature implementation.
Resource requirements	Re-	Very low. Fits quickly on CPU with negligible tuning cost.	Low. Slightly more tuning overhead than plain Linear Regression but still lightweight.	Moderate. CPU training is practical, but tuning and tree construction are noticeably heavier than linear baselines.
Interpretability		High. Coefficients provide a direct view of linear feature influence.	High to moderate. Coefficients remain interpretable, though shrinkage redistributes weight across correlated inputs.	Moderate to low. Feature importance can be estimated, but the ensemble decision process is less transparent than linear coefficients.
Scalability		High for simple baselines, though predictive power is limited when feature interactions are strongly non-linear.	High for correlated feature sets, with better numerical stability than plain Linear Regression.	High in predictive capacity and practical scalability, but tuning cost grows with the number of trees and design choices.

The regression comparison reinforces the same design pattern seen in the results chapter. Linear Regression is valuable as the simplest benchmark, Ridge Regression improves stability through regularisation, and XGBoost is the most expressive option because it captures non-linear feature interactions

that linear models cannot represent directly.

Table 3.4: Summary of reported rating-prediction performance

Model		RMSE	R^2	Within ± 0.5	Within ± 1.0	Interpretation
Linear	Regression	1.2451	-0.0547	51.14%	69.00%	Weak baseline; R^2 stays negative.
Ridge	Regression	1.0281	0.2809	33.08%	62.52%	Lower RMSE than the plain linear model.
XGBoost		0.5629	0.4196	79.42%	93.57%	Best regressor.

Tables 3.2 and 3.4 highlight two main outcomes. Among the directly comparable three-class sentiment models, **Linear SVC** offers the best balance between high test accuracy and stronger minority-class F1 values. Among the rating-prediction models, **XGBoost** is the strongest overall regressor because it produces the lowest RMSE and the clearest positive R^2 . The LSTM row remains useful as an auxiliary deep-learning reference, but it should be interpreted with the phase-2 comparability caveat noted above.

3.1.3 Algorithm Selection Justification

Linear SVC was selected for the sentiment-classification pipeline for four practical reasons:

1. **Optimal fit for sparse TF-IDF.** The feature matrix produced by TF-IDF is extremely high-dimensional and sparse (15,000+ features,

mostly zeros). Linear SVC is specifically designed for this regime. The LIBLINEAR back-end exploits sparsity and converges significantly faster than kernel SVMs or less specialised alternatives on large sparse matrices [3].

2. **Margin-based learning handles class boundaries better.** Unlike Logistic Regression, which minimises log-loss and can become over-confident near the decision boundary, Linear SVC maximises the geometric margin. For an imbalanced dataset where Neutral lies between Positive and Negative, this property produces more robust separation.
3. **Better Macro-F1 on minority classes.** Empirically, the tuned LinearSVC pipeline achieves better Negative and Neutral F1 values than Logistic Regression while preserving the same overall test accuracy. That is especially important because the project prioritises balanced performance over majority-class accuracy alone.
4. **Production-grade efficiency.** The full pipeline trains in a short time on CPU and does not require GPU infrastructure. This makes it the most practical final model among the directly comparable three-class sentiment candidates.

3.1.4 Regression Model Selection Justification

XGBoost was selected as the strongest rating-prediction model for the following reasons:

1. **Ability to model non-linear rating patterns.** User ratings are influenced by interacting textual and structured signals, and boosted trees can capture such non-linear relationships more effectively than purely linear regressors.
2. **Strong performance on mixed feature types.** The rating task combines review-derived signals with metadata, and XGBoost handles this heterogeneous feature mix naturally.

3. **Empirically strongest reported regressor.** Among the reported regressors, XGBoost achieves the best trade-off across RMSE, R^2 , and tolerance-based accuracy.
4. **Better fit than linear baselines.** Linear Regression is useful as a baseline and Ridge Regression improves stability, but neither matches the non-linear modelling flexibility demonstrated by XGBoost.

Chapter 4

Model-wise Analysis

4.1 Logistic Regression

Contributor: Shashikant Patel (IS24BM045)

4.1.1 Preprocessing and Implementation

The Logistic Regression pipeline uses cleaned review text as a sparse TF-IDF representation. The preprocessing removes HTML artefacts, normalises text, lowercases tokens for the bag-of-words representation, and avoids leakage by fitting the vocabulary only on the training split. The model is trained with class balancing and `GridSearchCV`, making it a strong and interpretable linear baseline for sentiment classification [2].

Table 4.1: Hyperparameter grid and rationale for Logistic Regression

Parameter	Rationale	Values
<code>C</code>	Controls regularisation. Smaller C increases regularisation and reduces overfitting; larger C fits training data more closely.	0.1, 1, 10
<code>penalty</code>	L2 regularisation distributes weight across correlated TF-IDF features and remains stable for sparse text.	l2
<code>solver</code>	<code>saga</code> is optimised for large sparse matrices; <code>liblinear</code> is efficient for smaller datasets.	<code>saga</code> , <code>liblinear</code>
<code>class_weight</code>	<code>balanced</code> compensates for the Positive-heavy class imbalance.	<code>balanced</code>

4.1.2 Results

The model achieves high accuracy across all splits, but the Neutral class remains the hardest to recover because its language overlaps with both polar classes.

Table 4.2: Logistic Regression accuracy across data splits

Split	Accuracy
Training	0.9961
Validation	0.9495
Test	0.9523

Table 4.3: Logistic Regression classification report on the test set

Class	Precision	Recall	F1-Score	Support
Negative	0.62	0.52	0.57	77
Neutral	0.44	0.28	0.34	74
Positive	0.97	0.99	0.98	2322
Macro avg	0.68	0.60	0.63	2473
Weighted avg	0.94	0.95	0.95	2473

Logistic Regression remains a reliable baseline, but its weakness is clear: the Positive-majority dataset lets the model retain high overall accuracy while the Neutral class still suffers from weak separation.

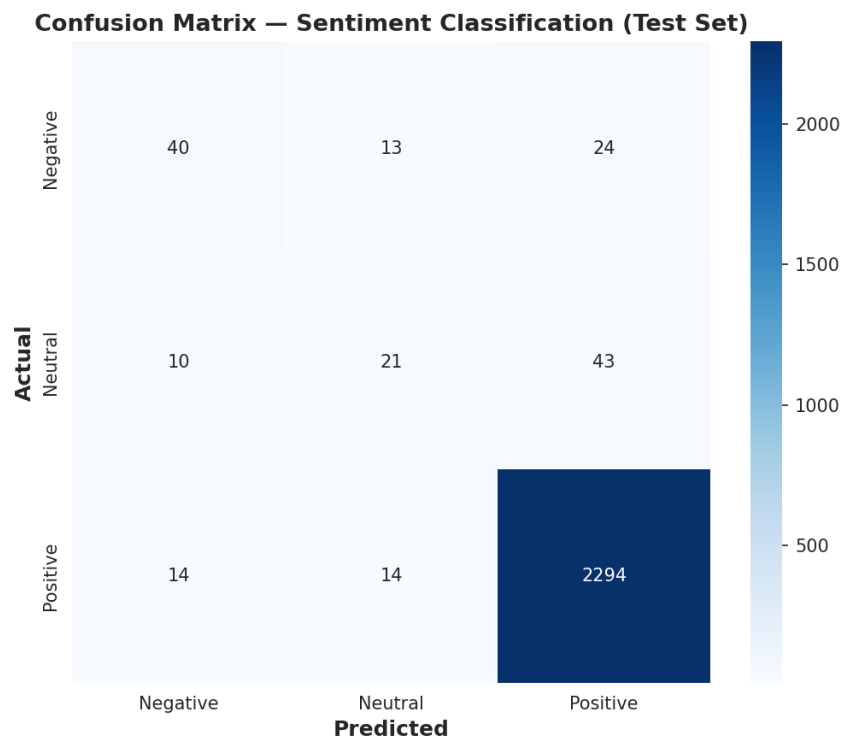


Figure 4.1: Logistic Regression confusion matrix

4.2 Linear SVC

Contributor: Skanda P (CS24BT045)

4.2.1 Preprocessing and Implementation

The Linear SVC pipeline contains the strongest feature engineering design in the report. It uses a dual-cleaning strategy: one stream normalises text for sparse lexical features, while another preserves casing and punctuation for sentiment-oriented meta-features. Instead of using vanilla TF-IDF alone, the final representation combines **delta TF-IDF** over 1–3 grams with **25 engineered meta-features**. After fusion, the final feature space contains **15,025 features** and is scaled with `MaxAbsScaler` to preserve sparsity. Hyperparameters are refined with Optuna-based tuning [6], making the full pipeline both CPU-friendly and highly effective for imbalanced sentiment classification [3].

Table 4.4: Summary of the feature engineering pipeline used for Linear SVC

Component	Role in the final SVM representation
Delta TF-IDF (1–3 grams)	Produces a sparse lexical representation and boosts class-discriminative tokens, especially terms useful for the Neutral class.
VADER and TextBlob scores	Add dense sentiment cues that complement bag-of-words features when polarity is explicit but lexical overlap is weak.
Linguistic and stylistic counts	Encode verbs, adjectives, adverbs, negations, punctuation intensity, lexical richness, and review length.
Sentence-level sentiment dynamics	Capture how sentiment shifts across the review using first-sentence, last-sentence, and sentence-level VADER statistics.
Intent and emphasis features	Flags such as <code>positive_intent_flag</code> , <code>negative_intent_flag</code> , and <code>caps_ratio</code> model recommendation intent and emphasis.

Table 4.5: Grouped view of the 25 engineered meta-features used in the SVM model

Group	Count	Features
Sentiment scores	6	vader_pos, vader_neu, vader_neg, vader_compound, textblob_polarity, textblob_subjectivity
Lexical and syntactic counts	11	verb_count, adj_count, adv_count, pos_keyword_count, neg_keyword_count, negation_density, word_count, exclamation_count, question_count, type_token_ratio, sentence_count
Sentence-level and intent features	8	sentence_vader_std, vader_contrast, last_sentence_vader, first_sentence_vader, negative_intent_flag, positive_intent_flag, caps_ratio, avg_sentence_length

4.2.2 Results

The tuned Linear SVC is the strongest directly comparable sentiment model in the report. Its main gain is not raw accuracy alone, but the improved balance it achieves on the minority Negative and Neutral classes.

Table 4.6: Test-set performance summary for SVM and baseline models

Model	Accuracy	Macro-F1
Majority-Class Baseline	0.9389	0.3228
VADER Rule-Based	0.8937	0.4745
Linear SVC — Untuned	0.9531	0.6310
Linear SVC — Tuned	0.9531	0.6439

Table 4.7: Linear SVC split-wise performance before and after tuning

Model	Split	Accuracy	Macro-F1	Neg F1	Neu F1	Pos F1
Untuned	Train	0.9986	0.9921	0.9944	0.9828	0.9993
	Validation	0.9482	0.5775	0.5147	0.2407	0.9770
	Test	0.9531	0.6310	0.6056	0.3077	0.9797
Tuned	Train	0.9938	0.9639	0.9550	0.9398	0.9968
	Validation	0.9527	0.6196	0.5652	0.3148	0.9787
	Test	0.9531	0.6439	0.6104	0.3423	0.9789

Table 4.8: Optuna-tuned Linear SVC classification report on the test set

Class	Precision	Recall	F1-score	Support
Negative	0.6104	0.6104	0.6104	77
Neutral	0.5135	0.2568	0.3423	74
Positive	0.9712	0.9866	0.9789	2322
Accuracy	0.9531			
Macro-F1	0.6439			

The tuned model keeps the same overall test accuracy as the untuned model, but it improves Macro-F1 by making better decisions on the minority classes. This is exactly where the delta TF-IDF and meta-feature design provides value.

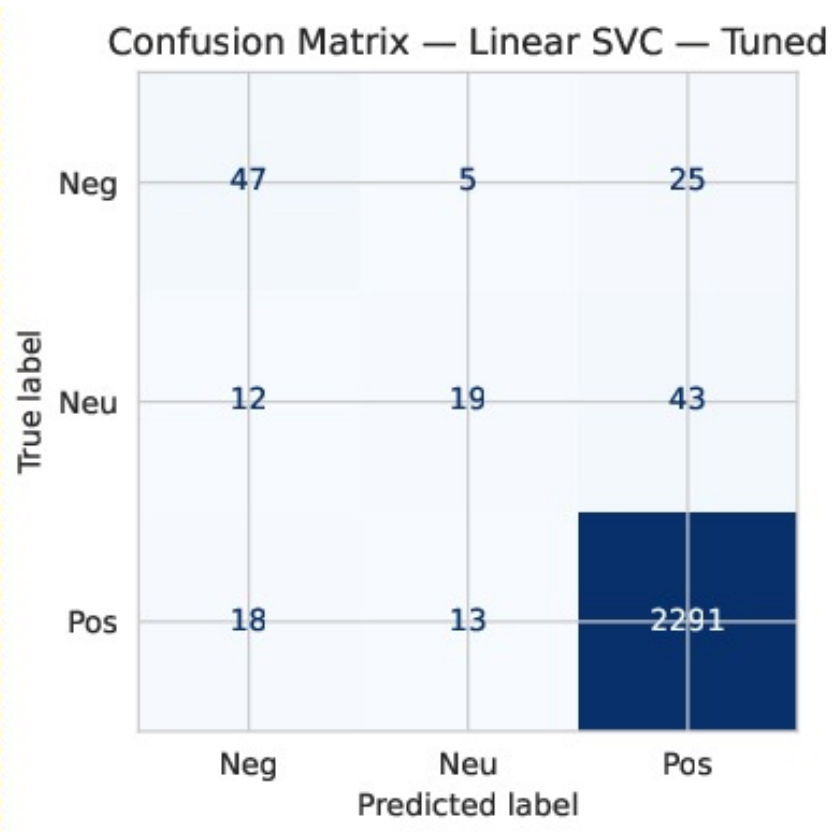


Figure 4.2: Linear SVC confusion matrix

4.3 LSTM with Word Embeddings

Contributor: Tanveer Rafiq Shaik (MC24BT014)

4.3.1 Preprocessing and Implementation

The LSTM section provides a sequence-aware alternative to the sparse linear models. Instead of relying primarily on TF-IDF, it learns from ordered token sequences through a BiRNN/LSTM-style architecture with word embeddings [4]. To mitigate class imbalance and noisy Neutral labels, the implementation follows a two-phase strategy:

- **Phase 1:** train on Positive and Negative reviews, then use the model to evaluate Neutral reviews and reassign high-confidence samples.
- **Phase 2:** retrain on the relabelled dataset and use the second model for evaluation and analysis.

This design attempts to preserve the benefit of sequence modelling while reducing the noise that makes the Neutral class difficult.

4.3.2 Results

The reported LSTM results are promising, especially in terms of Macro-F1 and balanced accuracy, but the evaluation setup is less directly comparable than the shared three-class LR/SVC pipeline.

Table 4.9: Classification report for LSTM

Class	Precision	Recall	F1-score	Support
Negative	0.49	0.48	0.49	92
Positive	0.98	0.98	0.98	2378
Macro Avg	0.74	0.73	0.73	2470
Weighted Avg	0.96	0.96	0.96	2470

Table 4.10: Performance metrics for LSTM

Metric	Value
Macro F1	0.7333
Balanced Accuracy	0.7297

The main takeaway is that preserving word order can help when sentiment depends on phrase structure or contrastive wording, but the sequence model is also more data-hungry and harder to stabilise under severe class imbalance.

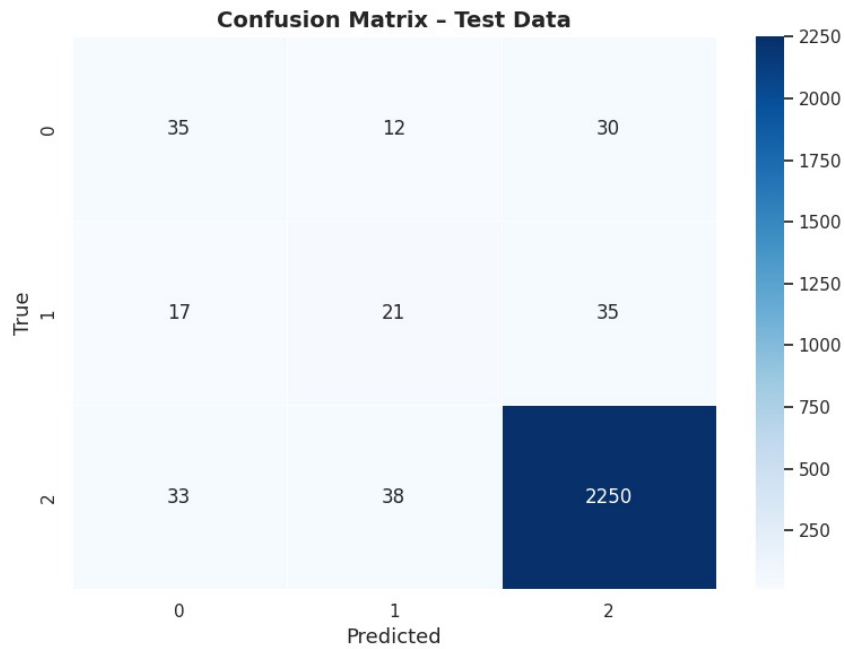


Figure 4.3: LSTM confusion matrix

4.4 XGBoost

Contributor: Krish Gupta (CS24BT038)

4.4.1 Preprocessing and Implementation

XGBoost is used for the rating-prediction task because it handles structured and mixed feature sets naturally and can model non-linear interactions between review text signals and metadata [7]. Compared with linear regressors, it is better suited to heterogeneous tabular inputs and does not require the relationship between features and rating to remain purely linear.

4.4.2 Results

Table 4.11 summarises the reported performance of XGBoost on the rating-prediction task. The model achieves strong tolerance- based accuracy and the best overall RMSE/ R^2 trade-off among the reported regressors.

Table 4.11: Reported XGBoost performance for rating prediction

Metric	Value
Accuracy (± 0.5)	79.42%
Accuracy (± 1.0)	93.57%
Mean Squared Error (MSE)	0.3169
Root Mean Squared Error (RMSE)	0.5629
R^2 Score	0.4196

The results show that boosted trees capture the non-linear structure of the rating task more effectively than the simpler baselines. This flexibility is what makes XGBoost the strongest reported regressor in the study.



Figure 4.4: XGBoost confusion matrix

4.5 Linear & Ridge Regression

Contributor: Palakurthy Shyam Praveen (CS24BT039)

4.5.1 Preprocessing and Implementation

The final model section groups the two linear rating baselines together because they use the same broad feature inputs but differ in how they control model complexity. Linear Regression provides the simplest reference point, while Ridge Regression adds L2 regularisation to stabilise coefficients when the feature set is correlated or noisy [2].

4.5.2 Results

Table 4.12 compares the two baselines directly. The plain Linear Regression model produces the slightly better tolerance-based accuracy, whereas Ridge Regression reduces overall error and achieves a positive R^2 , making it the stronger regularised baseline.

Table 4.12: Performance comparison of Linear and Ridge Regression					
Model	MAE	RMSE	R^2	Within $\pm 0.5\times$	Within $\pm 1.0\times$
Linear	0.8010	1.2451	-0.0547	0.5114	0.6900
Ridge	0.8399	1.0281	0.2809	0.3308	0.6252

Linear Regression. Linear Regression is useful as the simplest rating baseline because it shows how far a plain linear relationship can go before regularisation or non-linearity becomes necessary. Its tolerance-based accuracy is slightly better, but the negative R^2 shows that it is unstable overall.

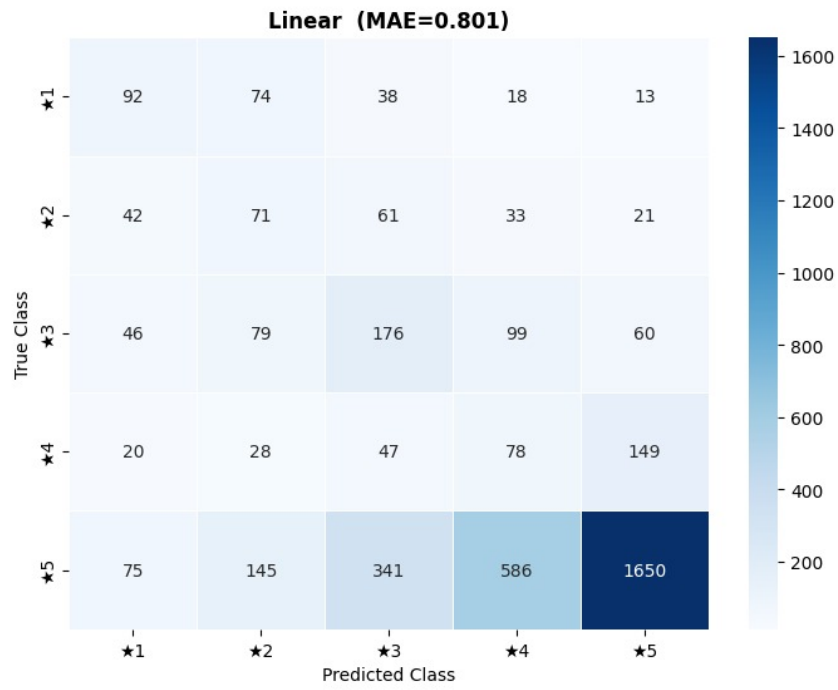


Figure 4.5: Linear Regression confusion matrix

Ridge Regression. Ridge Regression sacrifices some tolerance-based accuracy, but it reduces RMSE and achieves a positive R^2 . This indicates a more stable fit once regularisation is introduced.

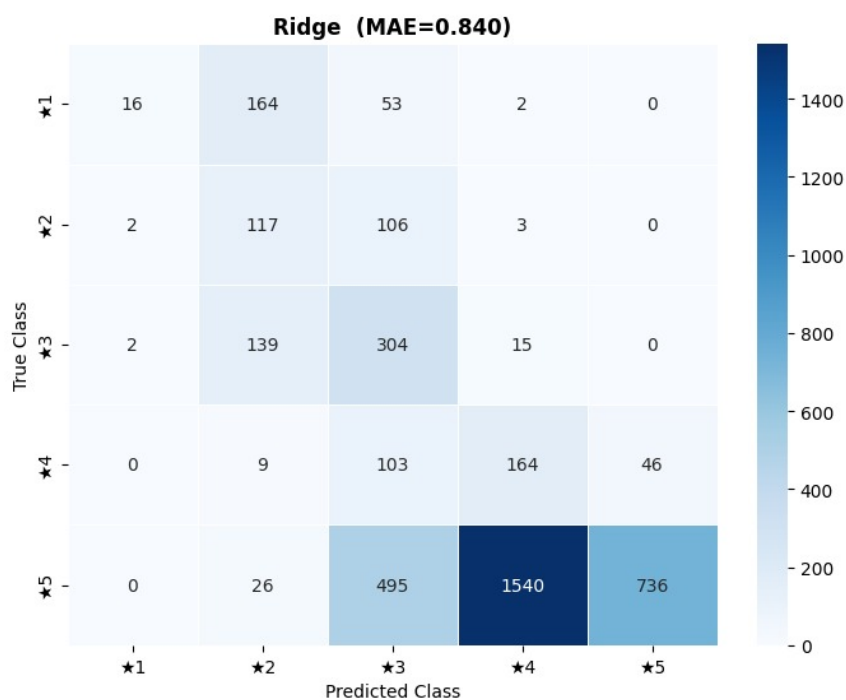


Figure 4.6: Ridge Regression confusion matrix

Chapter 5

Conclusion

This revised report gives greater emphasis to the experimental results and to the feature engineering that made the SVM pipeline effective. The most important technical contribution is not only the use of Linear SVC, but the combination of **delta TF-IDF** with **25 structured meta-features** that explicitly model polarity, subjectivity, negation, sentence-level sentiment shifts, and recommendation intent.

The final model choice is **Linear SVC**. Its tuned version improves Macro-F1 from 0.6310 to 0.6439 while keeping test accuracy at 0.9531 and strengthening the minority-class results that matter most in this dataset. Logistic Regression remains a useful baseline, and the LSTM/BiRNN results show that deep sequence models can also perform well, but the SVM pipeline provides the best overall balance of performance, interpretability, efficiency, and reproducibility.

For rating prediction, the uploaded materials indicate that **XGBoost** is the strongest overall regressor, with Ridge Regression outperforming the plain linear baseline among the explicitly reported linear models.

References

- [1] Dua, D. and Graff, C., “UCI Machine Learning Repository – Recipe Reviews and User Feedback Dataset,” University of California, Irvine, 2019.
- [2] F. Pedregosa, G. Varoquaux, A. Gramfort, *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [3] R.-E. Fan, K.-W. Chang, C.-J. Hsieh, X.-R. Wang, and C.-J. Lin, “LIBLINEAR: A Library for Large Linear Classification,” *Journal of Machine Learning Research*, vol. 9, pp. 1871–1874, 2008.
- [4] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [5] C. J. Hutto and E. Gilbert, “VADER: A Parsimonious Rule-Based Model for Sentiment Analysis of Social Media Text,” in *Proceedings of the Eighth International AAAI Conference on Weblogs and Social Media*, 2014.
- [6] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A Next-generation Hyperparameter Optimization Framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 2623–2631, 2019.

- [7] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.